

Reviewer Pack — Minimal Rank of Tropical Motivic Theory and DPLL Proof Depth

Entry b2a8aa0ec52a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 15:36:31 UTC

Minimal Rank of Tropical Motivic Theory and DPLL Proof Depth

Entry ID: b2a8aa0ec52a **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 15:36:31 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Motivic Theory) **Field B** (complexity object): DPLL Search Trees

Statement:

For every conjunctive normal form (CNF) with n variables, the minimal rank of its associated tropical motivic theory is linearly correlated with its DPLL proof depth, such that the minimal rank is $\Theta(2^n / n^2)$

Rationale (proposer's reasoning):

Tropical motivic theory has recently been applied to study properties of algebraic varieties and their associated invariants. The minimal rank provides a measure of complexity in tropical geometry. By exploring this invariant's relationship with the DPLL proof depth, we may uncover new insights into the complexity of solving SAT problems.

Taxonomy category: TROPICAL_MOTIVIC_THEORY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6d455c1ee6a08c0c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each CNF with n variables, if the correlation coefficient between the minimal rank of its associated tropical motivic theory and its DPLL proof depth is greater than or equal to 0.7 and the p-value is less than 0.05 using 30 seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropical geometry" AND "motivic theory" AND "DPLL search trees" - "minimal rank" [TI] AND "tropical motivic theory" AND "DPLL proof depth" - "CNF" [TI] AND $\Theta(2^n / n^2)$ AND "tropical geometry"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix, b):
    n = len(matrix)
    augmented_matrix = [row + [b[i]] for i, row in enumerate(matrix)]

    for i in range(n):
        # Find the pivot
        max_row = i
        for j in range(i+1, n):
            if abs(augmented_matrix[j][i]) > abs(augmented_matrix[max_row][i]):
                max_row = j

        # Swap rows
        augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]

        # Eliminate non-pivot elements in the current column
        for j in range(n):
            if i != j:
                factor = Fraction(augmented_matrix[j][i], augmented_matrix[i][i])
                for k in range(n + 1):
                    augmented_matrix[j][k] -= factor * augmented_matrix[i][k]

    # Check if the system has a unique solution
    rank = 0
    for row in augmented_matrix:
        if any(row[i] != 0 for i in range(n)):
            rank += 1

    return rank
```

```

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    min_ranks = []
    proof_depths = []

    for n in n_values:
        for _ in range(5): # Sample 5 instances per size
            cnf = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
            rank = gaussian_elimination(cnf, [random.randint(0, 1) for _ in range(n)])
            min_ranks.append(rank)

            # Simulate DPLL proof depth (placeholder)
            proof_depth = random.randint(1, n * n)
            proof_depths.append(proof_depth)

    if len(min_ranks) < 30 or len(proof_depths) < 30:
        return {
            "metric_name": "Correlation",
            "metric_value": None,
            "instances_tested": len(min_ranks),
            "n_max": max(n_values),
            "conjecture_holds": False,
            "counterexample": "Insufficient instances"
        }

    mean_min_rank = sum(min_ranks) / len(min_ranks)
    mean_proof_depth = sum(proof_depths) / len(proof_depths)

    correlation_coefficient = (n_values[-1] * sum(xi * yi for xi, yi in zip(min_ranks, proof_depths)) -
                               sum(min_ranks) * sum(proof_depths)) / \
        math.sqrt((n_values[-1] * sum(xi**2 for xi in min_ranks) - sum(min_ranks)**2) *
                  (n_values[-1] * sum(yi**2 for yi in proof_depths) - sum(proof_depths)**2))

    p_value = 2 * (1 - math.erf(abs(correlation_coefficient) / math.sqrt(2 * len(n_values))))

    return {
        "metric_name": "Correlation",
        "metric_value": correlation_coefficient,
        "instances_tested": len(min_ranks),
        "n_max": max(n_values),
        "conjecture_holds": correlation_coefficient >= 0.7 and p_value < 0.05,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...run_trial output...}}")
        results.append(trial_result)

```

```

mean_metric_value = sum(result["metric_value"] for result in results if result["metric_value"])
std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value)**2 for result in results) / len(results))
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif sum(1 for result in results if not result["conjecture_holds"]) / len(results) >= 0.2:
    print("RESULT: FALSIFIED counterexample=\"Insufficient evidence\" first_failing_seed=<s>")
else:
    print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a64e4541.py", line 99, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a64e4541.py", line 57, in run_trial
    rank = gaussian_elimination(cnf, [random.randint(0, 1) for _ in range(n)])
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a64e4541.py", line 35, in gaussian_elimination
    factor = Fraction(augmented_matrix[j][i], augmented_matrix[i][i])
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/fractions.py", line 281, in __new__
    raise ZeroDivisionError('Fraction(%s, 0)' % numerator)
ZeroDivisionError: Fraction(0, 0)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevented the production of data necessary to evaluate the conjecture. | next: Review and debug the test code to ensure it can run to completion without errors.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	21042
2	propose	ollama_remote	glm4:latest	0	0	17302
3	propose	ollama_remote	glm4:latest	0	0	22222
4	preregistration	ollama_remote	glm4:latest	0	0	9163
5	novelty	ollama_remote	glm4:latest	0	0	9650
6	novelty	ollama_remote	glm4:latest	0	0	16510
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20371
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15714
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16895
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12853
11	judge	ollama_remote	glm4:latest	0	0	19106

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 180828 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/b2a8aa0ec52a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b2a8aa0ec52a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b2a8aa0ec52a.tar.gz` (if generated)